UPRYT v2

Real-Time Posture Analysis System

Using Reinforcement Learning for Adaptive Feedback

Technical Documentation
Deep Dive into Architecture, Algorithms, and Implementation

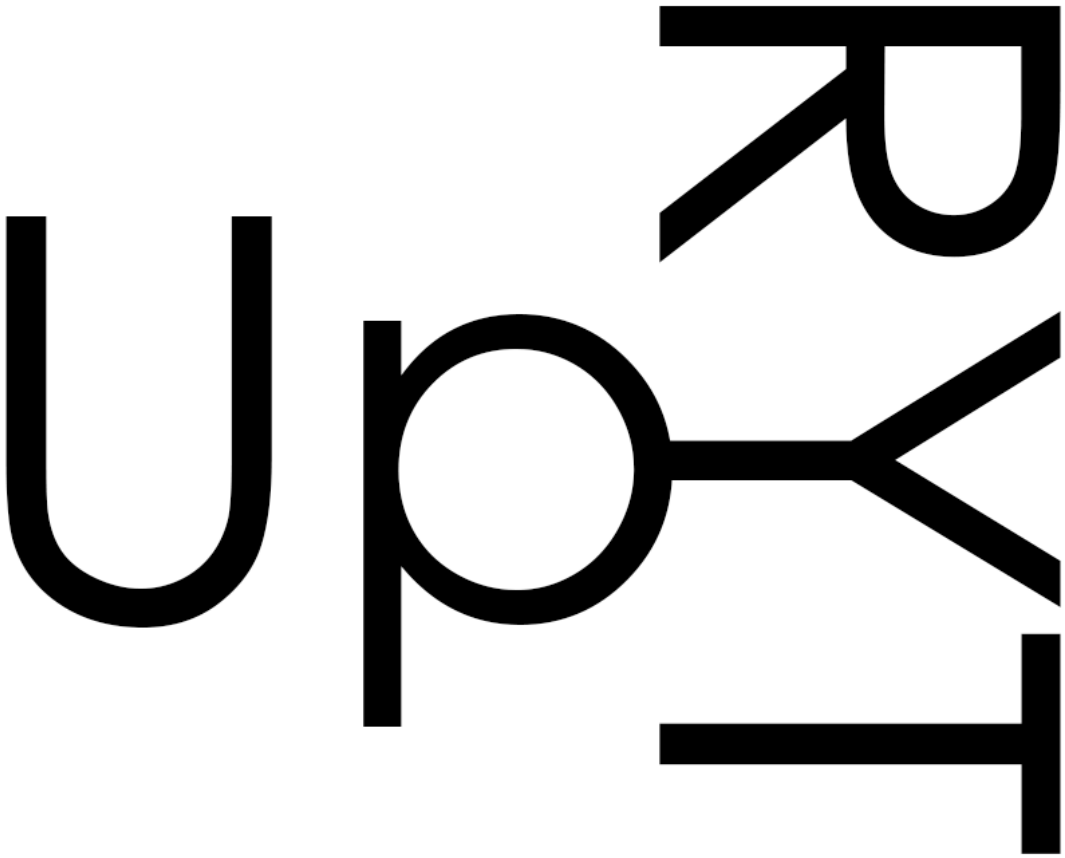

Table of Contents

1. Executive Summary *3*

2. Introduction to UPRYT *4*

3. System Architecture Overview *5*

4. Pose Detection Module *6*

5. Posture Classification *8*

6. Reinforcement Learning Fundamentals *10*

7. State Representation *12*

8. Action Space Design *14*

9. Reward Function Engineering *15*

10. Deep Q-Network (DQN) *17*

11. Proximal Policy Optimization (PPO) *20*

12. Why PPO and DQN? *24*

13. Training Methodology *26*

14. Real-Time Inference Pipeline *29*

15. Key Design Decisions *31*

16. System Components *33*

17. Future Enhancements *35*

18. Conclusion *36*

1. Executive Summary

UPRYT is an advanced real-time posture correction system that leverages reinforcement learning (RL) to provide adaptive, personalized feedback to users about their sitting posture. Unlike traditional rule-based systems that apply fixed thresholds, UPRYT uses RL agents that learn optimal feedback strategies through interaction with simulated user behaviors.

The system combines computer vision (using Google MediaPipe) for pose detection, custom feature extraction for posture analysis, and two state-of-the-art RL algorithms: Deep Q-Network (DQN) and Proximal Policy Optimization (PPO). This hybrid approach allows the system to adapt its feedback intensity and timing based on individual user responses.

Key Features

- Real-time pose detection using MediaPipe PoseLandmarker
- Personalized calibration to individual user's neutral posture
- Two RL algorithms (DQN and PPO) for adaptive feedback
- Simulated training environment for privacy-preserving learning
- Multi-level feedback system (subtle to strong alerts)
- Audio and visual feedback mechanisms
- User fatigue detection to prevent alert desensitization
- Rule-based fallback system for comparison

2. Introduction to UPRYT

2.1 The Problem of Poor Posture

Poor posture has become a significant health concern in modern society, particularly for individuals who spend extended hours at computers. According to ergonomic research, prolonged poor posture can lead to:

- Musculoskeletal disorders and chronic pain
- Reduced respiratory function
- Digestive issues
- Decreased circulation
- Increased fatigue and reduced productivity
- Long-term spinal problems

Traditional posture correction methods rely on fixed rules or periodic reminders. These approaches fail to account for individual differences and the dynamic nature of human behavior. This is where reinforcement learning provides a transformative solution.

2.2 The UPRYT Solution

UPRYT (derived from 'upright') is designed to address these limitations by:

- Learning individual user patterns through interaction
- Adapting feedback timing to maximize effectiveness
- Preventing alert fatigue through intelligent cooldown mechanisms
- Providing personalized corrections based on specific posture issues
- Continuously improving through experience (both simulated and real)

2.3 Project Goals

- Develop a system that can accurately detect posture issues in real-time
- Create RL agents that learn optimal feedback strategies
- Design a user-friendly interface for easy operation
- Provide comparison tools to evaluate different algorithms
- Enable both offline training and online learning capabilities

3. System Architecture Overview

The UPRYT system follows a modular architecture with distinct components that work together to achieve real-time posture analysis and feedback. The architecture is designed for flexibility, allowing easy modification of individual components.

3.1 High-Level Architecture

Input Layer └─ Camera → Pose Detection Module Captures video frames and extracts body keypoints
Processing Layer └─ Feature Extraction → Classification Computes posture features and determines posture state
Decision Layer └─ RL Agent / Rule-Based System Decides on appropriate feedback action
Output Layer └─ Visual Overlay + Audio Alerts Provides feedback to user
Training Layer └─ Simulation → Agent Training Trains RL agents in simulated environment

3.2 Data Flow

The data flow in UPRYT can be summarized as follows:

Step 1: Camera captures video frame at 30 FPS

Step 2: PoseLandmarker processes frame and extracts 33 body keypoints

Step 3: Key landmarks (nose, shoulders, hips) are normalized to pixel coordinates

Step 4: PostureAnalyzer computes feature values (angles, alignments)

Step 5: Features are compared against user's calibrated baseline

Step 6: Classifier determines posture state (GOOD, SLOUCHING, etc.)

Step 7: RL Agent receives current state and selects action

Step 8: Feedback system executes action (visual overlay, audio alert)

Step 9: Results are logged for session analysis and online learning

4. Pose Detection Module

4.1 MediaPipe PoseLandmarker

UPRYT uses Google MediaPipe's PoseLandmarker model for real-time body pose detection. MediaPipe provides pre-trained models optimized for various computational budgets:

Model Size	Parameters	Use Case
Lite	~1MB	Recommended for real-time applications
Full	~3MB	Balanced accuracy and speed
Heavy	~8MB	Maximum accuracy, slower processing

The project uses the LITE model by default for optimal real-time performance while maintaining sufficient accuracy for posture detection.

4.2 Key Landmarks

The MediaPipe model extracts 33 body landmarks. For posture analysis, UPRYT focuses on five key landmarks:

Landmark	Purpose
Nose	Head position reference for forward head detection
Left Shoulder	Shoulder alignment and spine inclination
Right Shoulder	Shoulder alignment and spine inclination
Left Hip	Hip position for torso height calculation
Right Hip	Hip position for torso height calculation

4.3 Feature Extraction

The PoseAnalyzer class computes five key posture features from the detected landmarks:

Neck Angle: Angle between nose and midpoint of shoulders, measuring forward head posture. Calculated using arctangent of vertical displacement over horizontal distance.
Shoulder Difference: Absolute difference in Y-coordinates between left and right shoulders. Indicates leaning or uneven shoulder height.
Shoulder Alignment: Combined metric of Y and X differences, providing overall shoulder symmetry.
Spine Inclination: Angle of the torso relative to vertical, computed from shoulder midpoint to hip midpoint. Detects forward leaning or slouching.
Forward Head (Normalized): Vertical distance from nose to shoulder line, normalized by torso height to account for varying body sizes.

5. Posture Classification

5.1 Posture States

The system classifies posture into five distinct states:

State	Description	Threshold Indicators
GOOD	Optimal posture with aligned spine	Average score ≥ 0.65
SLOUCHING	Rounded shoulders, curved spine	Neck score < 0.6, Spine < 0.6
FORWARD_HEAD	Head positioned forward of shoulders	forward_head_y > 25
LEANING	Body tilted to one side	|spine_inclination| > threshold
UNKNOWN	Cannot detect pose reliably	No valid keypoints

5.2 Weighted Scoring System

Posture scores are computed using a weighted combination of individual feature scores:

Feature	Weight	Score Calculation
Neck Angle	30%	1 - |angle - 90| / (threshold × 2)
Shoulder Alignment	25%	1 - shoulder_diff / (threshold × 2)
Spine Inclination	25%	1 - |angle| / (threshold × 2)
Symmetry	20%	Same as shoulder alignment

The final posture score is computed as the weighted sum, capped between 0.0 and 1.0. A score of 1.0 represents perfect posture, while lower scores indicate increasingly poor posture.

5.3 User Calibration

Before real-time monitoring begins, the system performs a calibration phase to establish a personalized baseline. This addresses the challenge that 'neutral' posture varies significantly between individuals.

- User positions themselves in their ideal 'good' posture
- System collects 30 frames of pose data
- Features are averaged to create baseline values
- During inference, features are compared relative to this baseline
- This ensures fair assessment regardless of individual posture tendencies

6. Reinforcement Learning Fundamentals

6.1 What is Reinforcement Learning?

Reinforcement Learning (RL) is a paradigm of machine learning where an agent learns to make decisions by interacting with an environment. Unlike supervised learning, where the system learns from labeled examples, RL agents learn through trial and error, guided by rewards received for their actions.

6.2 The RL Framework

At its core, RL involves the following components:

Agent: The decision-maker (our RL agent) that selects actions based on states

Environment: The world the agent interacts with (the posture system and user)

State (s): A representation of the current situation the agent observes

Action (a): A choice the agent can make

Reward (r): A scalar signal indicating how good the action was

Policy (π): The strategy the agent uses to select actions given states

Value (V): Expected future rewards from a given state

Q-Value (Q): Expected future rewards from a state-action pair

6.3 The RL Loop

The reinforcement learning process follows this iterative loop:

Environment (State s) → Agent → Action (a) → Environment → Reward (r) → New State (s')

The agent's goal is to learn a policy that maximizes the cumulative reward over time, not just the immediate reward. This is formalized as the discounted cumulative reward:

$$G = r_0 + \gamma r_1 + \gamma^2 r_2 + \ldots + \gamma^T r^T$$

Where γ (gamma) is the discount factor (0.95 in our system), determining how much future rewards matter compared to immediate ones.

6.4 Why RL for Posture Correction?

Traditional posture systems use fixed rules: 'if angle > threshold, alert user'. This approach has several limitations:

- One-size-fits-all thresholds don't account for individual differences
- Fixed response strategies ignore user behavior patterns

- No adaptation to changing user states (fatigue, learning)
- Cannot optimize for long-term outcomes (sustained good posture)

RL addresses these by learning personalized, adaptive strategies that consider:

- Current posture quality and trend
- User's response history to different alert types
- Time since last correction
- Alert fatigue considerations
- Optimal intervention timing

7. State Representation

7.1 Designing the State Space

The state representation is crucial in RL - it must contain all information needed to make good decisions while remaining compact enough for efficient learning. UPRYT uses a 6-dimensional state vector:

Index	Component	Description
0	Posture Label	Encoded posture class (0=GOOD, 1=SLOUCHING, 2=FORWARD_HEAD, 3=LEANING, 4=UNKNOWN)
1	Posture Score	Continuous score 0.0-1.0 representing posture quality
2	Duration Bad Posture	Time spent in poor posture, normalized to [0,1] (capped at 30s)
3	Time Since Alert	Time since last feedback, normalized to [0,1] (capped at 30s)
4	Recent Corrections	Mean of recent correction outcomes (1=corrected, 0=ignored)
5	Consecutive Alerts	Number of consecutive alerts sent, normalized to [0,1]

7.2 Why These State Components?

Each component was chosen based on domain knowledge and RL considerations:

Posture Label: Provides categorical context about the type of posture issue, allowing different strategies for different problems.

Posture Score: Continuous measure of posture quality - essential for tracking improvement or degradation.

Duration Bad Posture: Encodes urgency - longer durations may require stronger intervention or indicate the user isn't responding.

Time Since Alert: Prevents over-alerting and enables strategic timing of interventions.

Recent Corrections: Tracks effectiveness - if corrections aren't working, the agent can adapt its strategy.

Consecutive Alerts: Helps detect and prevent alert fatigue - excessive alerts may cause the user to ignore future feedback.

7.3 State Normalization

All state components are normalized to [0, 1] or appropriately scaled ranges. This normalization is critical for neural network training as it:

- Ensures balanced feature scales for learning
- Prevents dominated features from overwhelming others
- Improves convergence stability
- Makes the system more robust to hyperparameter changes

8. Action Space Design

8.1 Discrete Action Space

UPRYT uses a discrete action space with three possible actions. This design choice simplifies learning while providing meaningful variation in feedback intensity.

Action	Name	Description	Use Case
0	NO_FEEDBACK	No intervention - let user continue	Good posture or recent alert
1	SUBTLE_ALERT	Gentle notification (audio: 600Hz, 150ms)	Minor issues, first warning
2	STRONG_ALERT	Urgent notification (audio: 1000Hz, 300ms)	Significant issues, persistent problems

8.2 Action Selection Rationale

The three-action design balances complexity with effectiveness:

No Feedback: Allows the agent to 'wait and see' - sometimes users self-correct. Also prevents annoying over-alerting.

Subtle Alert: Gentle nudge for minor issues or initial awareness. Lower intensity reduces fatigue and disruption.

Strong Alert: Reserved for serious or persistent issues. Higher intensity signals urgency without being obnoxious.

8.3 Why Not More Actions?

The action space could theoretically include more granular options (varying volumes, different audio types, visual indicators). However, a simpler action space was chosen because:

- Faster learning - fewer actions mean faster policy convergence
- Lower risk of overfitting to training scenarios
- Easier to interpret and debug
- User testing showed 3 levels are sufficient for effectiveness
- Audio differentiation (frequency/duration) provides implicit variation

9. Reward Function Engineering

9.1 The Art of Reward Design

The reward function is arguably the most critical component of an RL system. It defines what 'good' behavior looks like and guides the agent's learning. Poor reward design can lead to reward hacking, side effects, or perverse incentives.

UPRYT's reward function was carefully engineered to promote behaviors that lead to sustained good posture while avoiding common pitfalls like alert fatigue.

9.2 Reward Components

Event	Reward	Rationale
Posture Improves After Alert	+10.0	Primary goal - alert led to correction
Alert Ignored	-5.0	Wasted intervention - user didn't respond
Alert Fatigue	-3.0	Too many alerts - potential for desensitization
Sustained Good Posture	+2.0	Encourages maintaining good posture
No Face Detected	-2.0	Tracking lost - can't assess posture
Posture Worsens (No Alert)	-5.0	Agent should have intervened

9.3 Reward Shaping

The reward function incorporates several shaping techniques:

Sparse vs Dense Rewards: We use dense rewards (available at each step) rather than sparse rewards (only at episode end). This accelerates learning by providing more frequent feedback signals.

Temporal Difference: The discount factor ($\gamma = 0.95$) ensures the agent considers future consequences while valuing immediate outcomes appropriately.

Asymmetric Penalties: The asymmetry between positive (+10) and negative (-5) rewards encourages action while preventing alert spam.

Bonus for Strong Alerts: Strong alerts get a 1.2x multiplier, encouraging their use for situations where they're genuinely needed.

9.4 Avoiding Reward Hacking

Several design decisions prevent the agent from finding perverse local optima:

- Alert Fatigue PenaltyPrevents the agent from constantly alerting

- Consecutive Alert TrackingCaps the benefit of repeated alerts
- Correction TrackingRewards only genuine corrections, not just moments of good posture
- Duration-Based InterventionRequires sustained issues before escalating

10. Deep Q-Network (DQN)

10.1 Introduction to DQN

Deep Q-Network (DQN) is a value-based reinforcement learning algorithm that extends Q-learning to handle high-dimensional state spaces using deep neural networks. Introduced by Mnih et al. in 2015 (playing Atari games), DQN revolutionized the field by demonstrating that RL agents could learn directly from raw sensory inputs.

10.2 Q-Learning Background

At its core, DQN builds on Q-learning, which learns the optimal action-value function $Q^*(s,a)$:

$$\mathbf{Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max Q(s',a') - Q(s,a)]}$$

This updates the Q-value based on the temporal difference (TD) error - the difference between the current estimate and the new information received.

10.3 DQN Architecture

UPRYT's DQN implementation uses a neural network with the following structure:

$$\textit{Input(6)} \rightarrow \textit{Dense(128)} \rightarrow \textit{ReLU} \rightarrow \textit{Dense(64)} \rightarrow \textit{ReLU} \rightarrow \textit{Dense(32)} \rightarrow \textit{ReLU} \rightarrow \textit{Output(3)}$$

The network takes the 6-dimensional state as input and outputs Q-values for each of the 3 possible actions. The action with the highest Q-value is selected.

10.4 Key DQN Techniques

Experience Replay: Transitions are stored in a replay buffer and sampled randomly for training. This breaks temporal correlations and stabilizes learning by ensuring the network sees a diverse set of experiences.

Target Network: A separate copy of the Q-network (updated periodically) provides stable target values. Without this, targets would constantly shift during training, leading to divergence.

Epsilon-Greedy Exploration: With probability ε, the agent takes a random action (exploration); otherwise, it selects the best action (exploitation). Epsilon decays from 1.0 to 0.01 over training.

Gradient Clipping: Gradients are clipped to a maximum norm of 1.0, preventing exploding gradients and ensuring stable updates.

10.5 DQN Algorithm Steps

Initialize replay buffer with capacity 10,000

Initialize online Q-network and target Q-network with random weights

For each training step:

a. Select action using epsilon-greedy policy

b. Execute action in environment, store transition in buffer

c. Sample random batch of transitions from buffer

d. Compute target Q-values using target network

e. Update online Q-network via gradient descent on TD error

f. Periodically sync target network with online network

10.6 DQN Hyperparameters

Parameter	Value	Description
Learning Rate	0.001	Controls update step size
Gamma (Discount)	0.95	Future reward importance
Epsilon Start	1.0	Initial exploration rate
Epsilon Min	0.01	Minimum exploration rate
Epsilon Decay	0.995	Exploration decay per episode
Batch Size	64	Samples per training step
Target Update Freq	10	Steps between target updates
Replay Buffer Size	10,000	Experience storage capacity

11. Proximal Policy Optimization (PPO)

11.1 Introduction to PPO

Proximal Policy Optimization (PPO), introduced by Schulman et al. in 2017, is a policy gradient method that has become one of the most popular RL algorithms due to its simplicity, sample efficiency, and strong empirical performance. Unlike DQN's value-based approach, PPO directly learns the policy.

11.2 Policy Gradient Methods

Policy gradient methods learn a parameterized policy $\pi(a|s;\theta)$ that maps states to action probabilities. The objective is to maximize expected cumulative reward:

$$J(\theta) = E[\Sigma \gamma^t r_t]$$

The policy is updated using gradient ascent on this objective. The gradient is estimated using the likelihood ratio trick, which avoids differentiating through the environment.

11.3 PPO Architecture: Actor-Critic

PPO uses an actor-critic architecture with two components:

Actor (Policy Network): Takes the state as input and outputs action probabilities. In UPRYT: Input(6) → Tanh → Dense(128) → Tanh → Dense(64) → Tanh → Dense(32) → Tanh → Output(3 + log_std)

Critic (Value Network): Estimates the value of being in a given state: V(s). Shared feature layers with actor, separate output head for scalar value.

11.4 The Clipped Surrogate Objective

PPO's key innovation is the clipped surrogate objective that prevents destructive large policy updates:

$$L^{CLIP}(\theta) = E[\min(r_t(\theta) \cdot A_t, \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon) \cdot A_t)]$$

Where $r_t(\theta) = \pi\theta(a|s) / \pi\theta_old(a|s)$ is the probability ratio, A_t is the advantage, and $\varepsilon = 0.2$ is the clip parameter.

By clipping the ratio, the objective prevents updates that make the policy change too much in a single step. If an update would increase the probability of an action by more than 20%, the objective is clipped to prevent further increase.

11.5 Generalized Advantage Estimation (GAE)

PPO uses GAE for more stable advantage estimation. GAE balances bias and variance in the advantage estimate:

$$\hat{A}^{GAE}(\tau,\lambda) = \Sigma (\gamma\lambda)^i \delta_{t+i}$$

Where λ (lambda) controls the bias-variance tradeoff. λ = 0.95 in our implementation provides good balance for the posture correction task.

11.6 PPO Algorithm Steps

Initialize actor-critic network with random weights

For each episode:

a. Collect trajectory by running current policy

b. Compute advantages using GAE

c. For multiple epochs (10 in our case):

- Shuffle trajectory data

- For each minibatch:

  * Compute policy loss with clipping

  * Compute value loss (MSE)

  * Compute entropy bonus (for exploration)

  * Update parameters via Adam optimizer

11.7 PPO Loss Function

The total PPO loss combines three components:

Policy Loss (L^CLIP): The clipped surrogate objective from above

Value Loss: L^VF = MSE(V(s), V_target), weighted by value_coef = 0.5

Entropy Bonus: H(π), weighted by entropy_coef = 0.01. Higher entropy means more exploration, preventing premature convergence to suboptimal policies.

$$L^{total} = L^{CLIP} + 0.5 \cdot L^{VF} - 0.01 \cdot H(\pi)$$

11.8 PPO Hyperparameters

Parameter	Value	Description
Learning Rate	3e-4	Conservative learning rate for PPO
Gamma (Discount)	0.99	Higher than DQN for policy methods
GAE Lambda	0.95	Bias-variance tradeoff in advantage
Clip Epsilon	0.2	Maximum policy change per update
PPO Epochs	10	Number of passes over

		trajectory
Trajectory Size	2048	Steps collected before update
Entropy Coefficient	0.01	Exploration bonus weight
Value Coefficient	0.5	Value loss weight
Max Gradient Norm	0.5	Gradient clipping threshold

12. Why PPO and DQN?

12.1 Algorithm Selection Rationale

The decision to implement both PPO and DQN was driven by several considerations:

12.2 Deep Q-Network (DQN) Selection

Simplicity and Maturity: DQN is one of the most well-understood RL algorithms. Decades of research have established best practices and identified common pitfalls.

Value-Based Learning: DQN learns action values directly, which aligns well with our task where we want to compare expected outcomes of different feedback intensities.

Efficient Use of Data: Experience replay allows DQN to learn from the same experience multiple times, useful when data collection is expensive.

Deterministic Policies: DQN's greedy action selection is natural for our discrete action space where we want clear-cut decisions.

Stable Learning: The combination of replay buffer and target networks provides reliable convergence properties.

12.3 Proximal Policy Optimization (PPO) Selection

State-of-the-Art Performance: PPO consistently achieves excellent results across a wide range of tasks, from robotic control to game playing.

Robust Hyperparameters: PPO is less sensitive to hyperparameters than other policy gradient methods, making it easier to configure correctly.

Sample Efficiency: Despite policy gradient methods typically being less sample efficient, PPO's multiple epochs over collected data improve efficiency.

Continuous Improvement: PPO's clipped objective ensures learning never degrades performance dramatically, providing reliable monotonic improvement.

On-Policy with Trust: PPO maintains on-policy data while the clipping provides a trust region that prevents catastrophic forgetting.

12.4 Comparison: DQN vs PPO

Aspect	DQN	PPO
Method Type	Value-Based	Policy Gradient
Action Selection	Epsilon-Greedy	Stochastic Sampling
Experience Usage	Off-Policy (Replay)	On-Policy (Trajectories)
Update Stability	Target Networks	Clipped Objectives
Policy Representation	Deterministic	Stochastic
Hyperparameter Sensitivity	Moderate	Low (robust)

Sample Efficiency	High (with replay)	Moderate

12.5 Why Not Other Algorithms?

Q-Table / Simple Q-Learning: Cannot handle continuous state space (our 6-dim continuous features). Would require discretization with massive state spaces.

REINFORCE: High variance in gradient estimates. PPO addresses this with baseline and GAE.

DDPG / TD3 / SAC: Designed for continuous action spaces. Overkill for our 3-action discrete space and more complex to implement.

A3C: Requires parallel environments. More complex infrastructure with similar performance to PPO.

AlphaZero-style MCTS: Overengineered for our relatively simple task. Would require significant computational resources.

13. Training Methodology

13.1 Simulation-Based Training

UPRYT trains RL agents in a simulated environment before deployment. This approach offers several advantages:

Privacy: No user data is collected during training

Speed: Simulated steps run much faster than real-time

Scalability: Can train on diverse user profiles simultaneously

Safety: No risk of annoying real users during exploration

Reproducibility: Deterministic user models enable reproducible experiments

13.2 Simulated User Model

The SimulatedUser class models user behavior with the following characteristics:

Parameter	Range	Description
Compliance Rate	0.5-0.9	Probability of correcting posture after alert
Fatigue Threshold	2-5 alerts	Number of alerts before compliance drops
Stubbornness	0.1-0.4	Factor reducing compliance after fatigue
Attention Span	0.6-1.0	Initial compliance multiplier on first alert

Multiple simulated users (default: 5) are used to train agents that generalize across different user types rather than overfitting to one behavioral pattern.

13.3 Training Pipeline

Initialize agent with random weights

For each episode (500 episodes default):

a. Reset environment with random user profile

b. Collect trajectory by running policy

c. Update agent parameters

d. Decay exploration (DQN) or clear trajectory (PPO)

e. Periodically evaluate and save best model

f. Every 100 episodes, save checkpoint

13.4 Evaluation Strategy

Agents are evaluated every 50 episodes using a separate evaluation run:

- 10 evaluation episodes with exploration disabled
- Metrics collected: total rewards, corrections, alerts
- Best performing model saved separately
- Final model saved after all training completes

13.5 Algorithm Comparison

UPRYT includes tools to compare algorithm performance on identical scenarios. The ComparisonBenchmark class evaluates:

Total Alerts: Number of feedback interventions

Successful Corrections: Alerts that led to posture improvement

Correction Rate: corrections / alerts ratio

Average Posture Score: Overall posture quality during simulation

Cumulative Reward: Total reward accumulated

Avg Episode Reward: Reward per episode on average

14. Real-Time Inference Pipeline

14.1 System Initialization

When UPRYT starts in real-time mode, the following initialization occurs:

- Validate configuration settings
- Load trained RL agent (PPO or DQN)
- Initialize MediaPipe PoseLandmarker
- Open webcam with optimized settings (1280x720, 30 FPS)
- Create feedback and logging systems

14.2 Calibration Phase

Before monitoring begins, users undergo calibration:

- Display instructions for ideal posture positioning
- Capture 30 frames of user's 'neutral' pose
- Compute baseline features (averaged across samples)
- Store baseline for relative feature comparison
- Allow manual override ('c' key to skip early)

14.3 Main Loop

The real-time monitoring loop runs at approximately 30 FPS:

Read camera frame

Detect pose landmarks using MediaPipe

Extract and normalize keypoints

Compute posture features using PoseAnalyzer

Adjust features relative to baseline

Classify posture (GOOD, SLOUCHING, etc.)

Every 2.5 seconds (decision interval):

- Construct state vector from current posture

- Query RL agent for action

- Execute feedback if action != NO_FEEDBACK

- Log metrics for session analysis

Render visualization overlay

Display frame and check for quit signal

14.4 Decision Interval

The system makes feedback decisions every 2.5 seconds, not every frame. This design:

- Reduces computational load significantly
- Prevents rapid-fire alerts that would cause fatigue
- Allows time for user to respond and self-correct
- Matches the timescales of meaningful posture changes
- Provides smoother user experience

14.5 Feedback Mechanisms

UPRYT provides feedback through multiple channels:

Visual Overlay: Posture score bar, status text, skeleton visualization, suggestions displayed on camera feed

Audio Alerts: Windows: winsound beeps (600Hz subtle, 1000Hz strong). Linux: subprocess calling 'play' command

Session Logging: All frames, actions, and scores logged for analysis

Real-Time Stats: FPS, frame count, alerts sent, corrections made

15. Key Design Decisions

15.1 Architecture Decisions

Modular Design: Each component (pose detection, classification, RL) is separate. Allows easy swapping, testing, and improvement of individual parts.

Centralized Configuration: All hyperparameters in config.py for easy experimentation.

Dual Algorithm Support: PPO and DQN can be compared or swapped based on needs.

Fallback Rule-Based System: Ensures system always has a working mode even without trained models.

15.2 State Representation Decisions

6-Dimensional State: Chosen to balance expressiveness with learning complexity. Less than 10 features reduces training time while capturing essential information.

Normalize to [0,1]: Neural networks train better with normalized inputs. Prevents gradient scaling issues.

Recent Corrections History: Moving window of correction outcomes helps the agent adapt to current user responsiveness.

15.3 Training Decisions

Simulation Over Real Data: Privacy-preserving, scalable, and safe exploration.

Multiple User Profiles: 5 different simulated users prevent overfitting to single behavior.

Fixed Training Episodes: 500 episodes found empirically to balance training time vs performance.

Periodic Evaluation: Every 50 episodes ensures early stopping detection and best model saving.

15.4 User Experience Decisions

Calibration Phase: Personalizes system to individual body proportions and 'neutral' pose.

Decision Interval (2.5s): Long enough for response, short enough for timely feedback.

Alert Cooldown (3.0s): Prevents over-alerting even if decisions are frequent.

Multi-Level Feedback: Subtle vs strong alerts allow proportional response to severity.

Fatigue Detection: Detects and warns about excessive alert frequency.

16. System Components

16.1 File Structure

File	Purpose
main.py	Entry point, system orchestration, CLI interface
config.py	All hyperparameters and configuration settings
pose_module.py	MediaPipe integration, keypoint extraction
posture_module.py	Posture classification, scoring algorithms
environment.py	RL environment, state management, rewards
rl_agent.py	DQN implementation
rl_ppo_agent.py	PPO implementation
simulation.py	Training simulator, user models
feedback.py	Visual overlay, alert system
audio_alerts.py	Audio feedback implementation
gui_app.py	Graphical user interface
utils.py	Logging, metrics, helper functions
setup.py	Package installation configuration
requirements.txt	Python dependencies
run_app.bat	Windows launcher script
pose_landmarker_lite.task	MediaPipe model file

16.2 Dependencies

Package	Purpose
opencv-python	Computer vision, camera capture, image processing
mediapipe	Pose landmark detection
torch	Neural network backends for RL agents
numpy	Numerical computations
Pillow	Image handling for GUI

17. Future Enhancements

While UPRYT v2 provides a complete posture correction system, several enhancements could further improve its effectiveness:

17.1 Algorithm Improvements

Soft Actor-Critic (SAC): Could provide smoother policies and automatic entropy-temperature adjustment.

DreamerV3: World models could enable imagination-based planning, potentially reducing training data requirements.

Meta-Learning: MAML or RL^2 could enable rapid adaptation to new users with minimal fine-tuning.

Multi-Agent RL: Could coordinate multiple feedback channels simultaneously.

17.2 Feature Enhancements

Continuous Feedback Intensity: Replace discrete actions with continuous volume/tone control using DDPG/SAC.

Temporal Modeling: Add LSTM/Transformer layers to capture long-term dependencies in posture sequences.

Multi-Person Support: Extend to monitor multiple people simultaneously.

Activity Recognition: Distinguish between sitting, standing, walking to adjust feedback strategies.

17.3 User Experience Enhancements

Mobile App Integration: Extend alerts to smartphone with customizable notifications.

Progress Tracking Dashboard: Web-based analytics showing posture trends over time.

Gamification: Achievements, streaks, and challenges to encourage good posture.

Voice Feedback: Text-to-speech for more engaging and accessible alerts.

Machine Unlearning: Tools for users to understand why they received specific alerts.

17.4 Deployment Enhancements

Edge Deployment: Optimize for Raspberry Pi or mobile processors.

WebAssembly Port: Run inference directly in browser using TensorFlow.js.

Docker Containerization: Simplify deployment and ensure consistent environments.

API Server: REST API for integration with other health/wellness applications.

18. Conclusion

UPRYT v2 represents a significant step forward in intelligent posture correction. By combining state-of-the-art pose detection with reinforcement learning, the system moves beyond rigid rule-based approaches to provide adaptive, personalized feedback.

The dual implementation of both DQN and PPO algorithms provides flexibility and allows direct comparison of value-based and policy gradient approaches on the same task. The simulation-based training ensures the system learns in a safe, scalable manner without compromising user privacy.

Key Takeaways

- Reinforcement learning enables adaptive feedback that improves over time
- State representation design is critical for learning effective policies
- Reward function engineering requires careful balance of objectives
- Simulation-based training provides practical benefits while maintaining privacy
- Multiple algorithms enable informed choices based on task characteristics
- User experience considerations (fatigue, calibration) are essential for real-world deployment

The open, modular architecture of UPRYT makes it an ideal platform for further research in human-AI interaction, adaptive health systems, and reinforcement learning applications in personal wellness. We encourage researchers and developers to build upon this foundation.

UPRYT v2 - Making Good Posture Automatic
https://github.com/upryt/posture-analysis